

$$T(n) = a T\left(\frac{n}{b}\right) + f(n) \quad n^{\log_b a}$$

Case 1: Constant ϵ that $f(n) = O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$

Case 2: Constant $k \geq 2$ that $f(n) = \Theta(n^{\log_b a} \lg^k n)$ then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$

Case 3: Constant ϵ that $f(n) = \Omega(n^{\log_b a + \epsilon})$? regularity cond. at $\frac{a}{b}$ for constant $c < 1$ and at sufficiently large n , then $T(n) = \Theta(f(n))$

If dom. function dominates case 3

If undersized function dominates, case 1

If domination is only done logarithmically (not polynomially)

THEN case 2.

4.5.1

Use the master method to give tight asymptotic bounds for the following recurrences.

a. $T(n) = 2T(n/4) + 1$

b. $T(n) = 2T(n/4) + \sqrt{n}$

c. $T(n) = 2T(n/4) + \sqrt{n} \lg^2 n$

d. $T(n) = 2T(n/4) + n$

e. $T(n) = 2T(n/4) + n^2$

$$\left. \begin{array}{l} a=2 \\ b=4 \end{array} \right\} n^{\log_b a} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$$

a) $T(n) = \underbrace{2T(n/4)}_{f(n)} + 1$

$f(n) = \sqrt{n} \rightarrow T(n) = \Theta(\sqrt{n})$
WS dominates

b) $T(n) = \underbrace{2T(n/4)}_{f(n)} + \frac{\sqrt{n}}{f(n)}$

WS = $\sqrt{n}$ $f(n) = \sqrt{n}$

So: case 2 applies

$$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$$

$$T(n) = \Theta(\sqrt{n} \lg^2 n)$$

c) $T(n) = \underbrace{2T(n/4)}_{f(n)} + \underbrace{\sqrt{n} \lg^2 n}_{f(n)}$

WS = $\sqrt{n}$ $f(n) = \sqrt{n} \lg^2 n$

Case 2: $f(n)$ is larger but only by $\lg^k$ factor

(Almost exactly the case 2 example)

$$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$$

$$T(n) = \Theta(\sqrt{n} \lg^{(2)+1} n)$$

$$T(n) = \Theta(\sqrt{n} \lg^3 n)$$

d) $T(n) = \underbrace{2T(n/4)}_{\sqrt{n}} + \underbrace{n}_n$

Function clearly dominates,

$$f(n) = \Omega(n^{\log_b a + \epsilon}) \rightarrow \Omega(n^{1/2 + \epsilon}) \rightarrow \epsilon = 1/2, 1/2 > 0$$

$$T(n) = \Theta(f(n)) \quad a f(n/b) = 2 f(n/4) = \frac{a}{b}$$

$$T(n) = \Theta(f(n)) \quad \text{if } f(n/b) = c f(n/4) = \frac{1}{2}$$

$$T(n) = \Theta(n)$$

$$c) T(n) = 2T\left(\frac{n}{4}\right) + \frac{n^2}{\sqrt{n}}$$

$$n^2 \text{ dominates, so } T(n^{1/2+e}) \rightarrow 2 = \frac{1}{2} + e, \quad e = 3/2$$

$$af(\frac{n}{b}) = cf(\frac{n}{4}) = cT(\frac{n}{4}) = \frac{n^2}{8} \quad c = 1/8 \checkmark$$

$$T(n) = \Theta(f(n))$$

$$T(n) = \Theta(n^2)$$

2. What is the time complexity $T(n)$ of the nested loops below? For simplicity, you may assume that n is a power of 2. That is, $n = 2^k$ for some positive integer k .

```

1 = n;
while (i >= 1) {
  j = i;
  while (j <= n) {
    <body of the inner while loop > // Needs  $\Theta(1)$ .
    j = 2 * j;
  }
  i = i/2;
}

```

$$n = 2^{48}$$

$$\log_2 n$$

$$T(n) = O(\log_2 n \cdot \log_2 n) = O((\log_2 n)^2)$$

Each time the inner executes it runs $\log_2 n$ times, increase with the outer loop.

3. Use recursive binary search to search for the integer 100 in the following list (array) of integers. Show the actions step by step.

12 34 37 45 57 82 95 100 134

I assume you are looking for us to not write the code but instead show how it would search step by step, so I have done that here:

12 34 37 45 57 82 95 100 134

Array size = 9

middle = 9/2 = 4

12 34 37 45 57 82 95 100 134

AC: 57

100 > 57

search right half

$$(9-4)/2 + 4 = 6$$

57 82 95 100 134

AC: 95

100 > 95

search right half

$$(5-6)/2 + 6 = 7$$

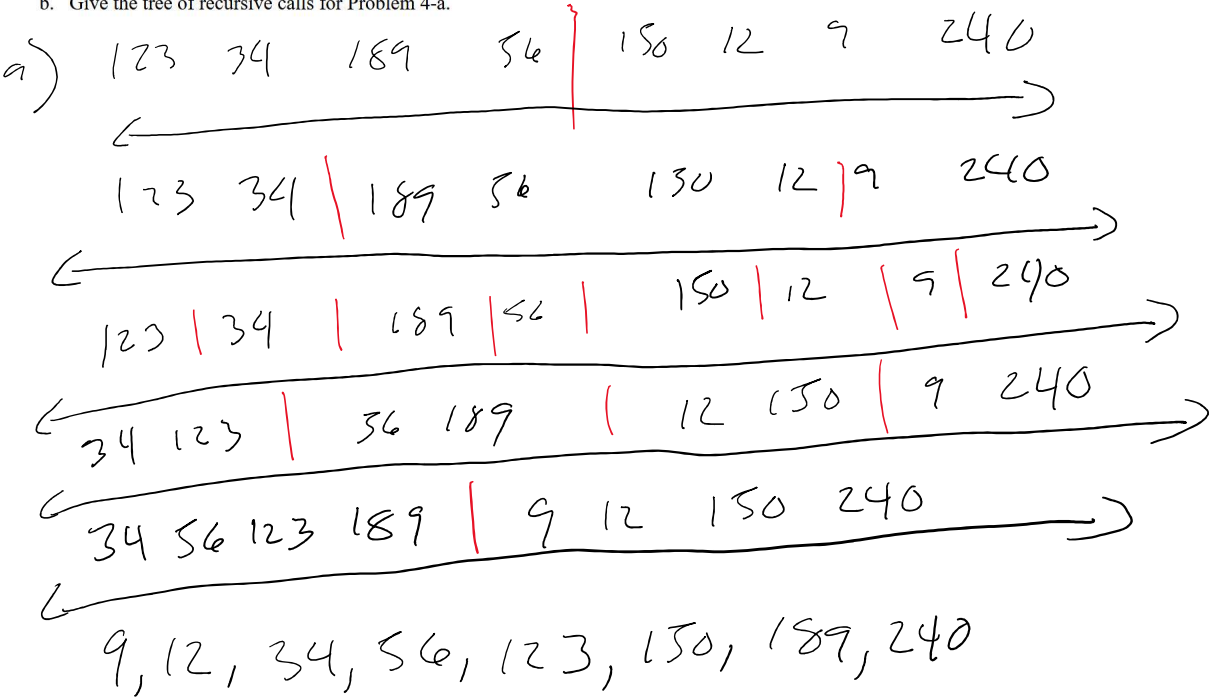
100 134

AC: 1 = 100
 100 = 100 ✓ (u)

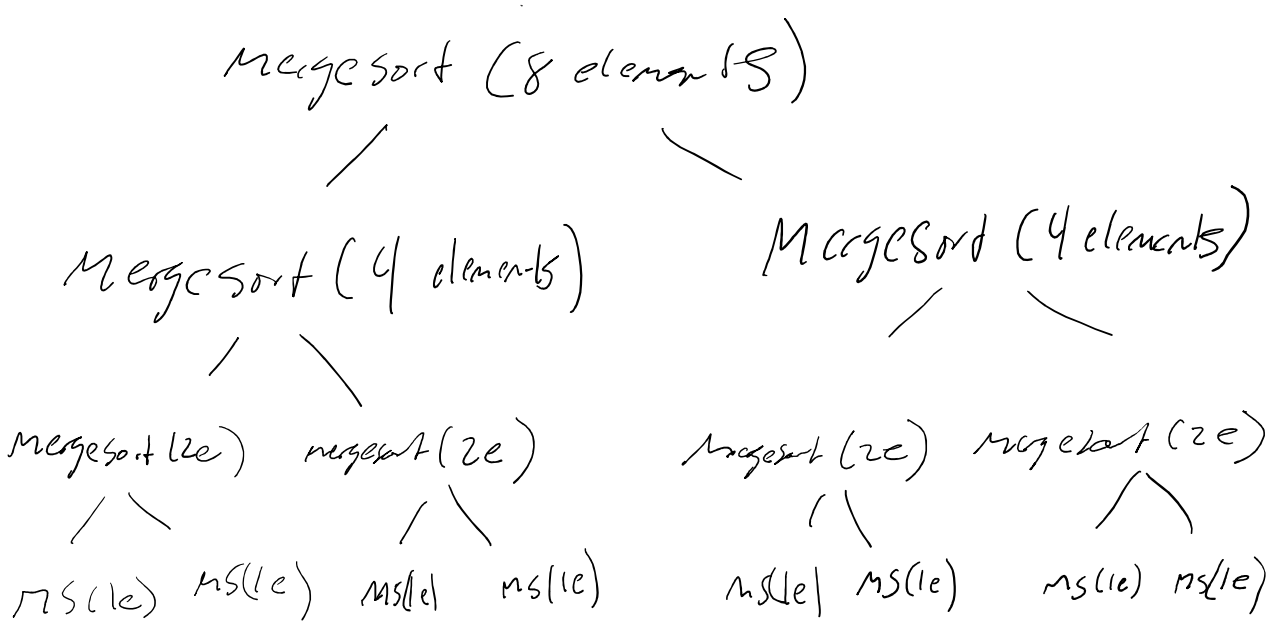
4. a. Use Merge sort to sort the following list. Show the action step by step.

123 34 189 56 150 12 9 240

b. Give the tree of recursive calls for Problem 4-a.



b) (Problem 4-a tree of calls)



Each part of the tree calls mergeSort, divides;
 then / merges arrays back together recursively

swap / merge arrays back together recursively

5. a. Suppose that we were to search a list of 100,000 items using binary search. What is the maximum number of comparisons that this algorithm must perform before finding a given item or concluding that it is not in the list?
- b. In general, if you were to perform a "ternary search" (by dividing the list into three approximately equal parts and eliminating two of the three thirds at "each level"), do you think you can obtain a better θ (theta) efficiency? Explain your rationale.

$$O(n) = \log_2(n)$$

$$O(100,000) = \log_2(100,000)$$

$$O(100,000) = 16.61$$

→

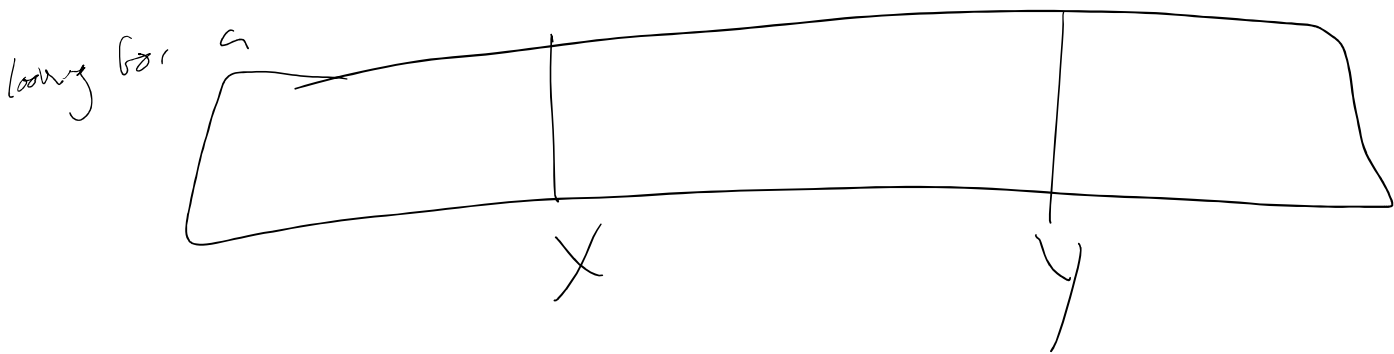
$$\underline{O(100,000) = 17.}$$

b) No, the list is already sorted and the amount of comparisons made per level could increase (1:2, or 1:2:3), even if the

level would increase (1:2, or 1:2:3), even if the number of splits would decrease.

Ternary in this case is still logarithmic, just $\log_3$ rather than $\log_2$ (for splits per level)

Worst case is in the middle, best case is all left/right.



$a > X$, $a > Y \rightarrow 2$ comparisons

$a < X$, $\rightarrow 1$ comparison

$a \geq X$, $a < Y \rightarrow 2$ comparisons

OR if backwards

$a > Y \rightarrow 1$ comparison

$a < Y$, $a > X \rightarrow 2$ comparisons

$a < Y$, $a < X \rightarrow 2$ comparisons